

NoSQL — Unit 2 Notes

Module 1–4: Distribution Models, Consistency, Version Stamps & Transactions, and Exam Preparation
(Based on NoSQL Distilled — Sadalage & Fowler, Chapters 4, 5 & 6)

MODULE 1: DISTRIBUTION MODELS

Why Distribution Matters

The main reason people adopt NoSQL is its ability to run on a **cluster** of machines instead of a single large server. As data volumes and traffic grow, scaling up (buying a bigger server) becomes expensive and eventually hits a physical ceiling. Distributing data across many commodity servers — **scaling out** — is cheaper and, in principle, has no upper limit.

There are **two orthogonal paths** to data distribution, and a system can use either one or both together:

Technique	What It Does
Replication	Takes the <i>same</i> data and copies it over multiple nodes, so each bit of data can be found in multiple places
Sharding	Puts <i>different</i> data on different nodes, so each server acts as the single source for a subset of the data

Exam Point: Sharding and replication are **orthogonal** — you can use one, the other, or combine both. Replication itself comes in two forms: **master-slave** and **peer-to-peer**.

Single Server

The simplest distribution model is **no distribution at all** — running the entire database on a single machine.

Despite all the excitement around clusters, a single-server model is often the right choice. It is easier for application developers because there is no need to worry about network partitions, replication lag, or sharding logic. Many document and graph databases run comfortably on a single server for small-to-medium workloads, and graph databases in particular are often naturally suited to single-server operation because distributed graph traversals are difficult to implement efficiently.

Real-world intuition: A small college's internal student-record system, used by a handful of staff, has no need for a distributed cluster — a single well-backed-up server is simpler, cheaper, and easier to maintain.

Exam Point: If you can avoid distributing your data, you should — a single-server setup is always the simplest option when it can handle the load.

Sharding

Often, a busy data store is busy because **different people are accessing different parts of the dataset**. In these cases, we can support horizontal scalability through **sharding** — putting different parts of the data onto different servers.

In the ideal case, each user talks to only one server, so the load is spread evenly. For example, with ten shards, each server only has to handle roughly 10% of the overall load.

Key considerations for effective sharding:

- **Data locality** — Data that is accessed together should be clustered together on the same shard, so a single request rarely needs to visit multiple shards.
- **Sharding strategy** — For example, customers with surnames A–D might go on one shard and E–G on another; or data can be sharded geographically so that users are served by the server nearest them.
- **Load balancing** — Shards should be arranged so that the load (not just the data volume) is spread evenly, since some records are accessed far more often than others.

Advantages and disadvantages of sharding:

Advantages	Disadvantages
Improves both read and write performance, since load is spread across servers	Offers no improvement in resilience by itself — if a shard's server goes down, that data becomes unavailable

Can reduce latency by placing data closer to the users who access it most	Complicates the programming model, since application code must route queries to the correct shard(s)
Scales horizontally as more servers can simply be added	Many NoSQL databases offer auto-sharding , taking this complexity away from the application; relational databases traditionally required manual sharding

Real-world intuition: An e-commerce platform might shard its product catalog by category (electronics on one shard, clothing on another) so that a spike in traffic for one category doesn't slow down searches in a completely unrelated category.

Exam Point: Sharding improves **performance** (read and write throughput) but, on its own, does **not** improve resilience — losing a shard's server still means losing access to that data. This is why sharding is often combined with replication.

Master-Slave Replication

With **master-slave replication**, data is replicated across multiple nodes, but **one node is designated as the master** (or primary). The master is the authoritative copy, responsible for processing all **writes**. The other nodes are **slaves** (or secondaries, or replicas), and a replication process keeps them synchronized with the master. **Reads** can be served from either the master or any of the slaves.

How a master is appointed:

Method	How It Works
Manually appointed	A node is configured as master when the cluster is set up; if it fails, an administrator (or a separate process) must appoint a new master
Automatically appointed	The cluster of nodes elects a master among themselves (often using a consensus process), and can elect a new one automatically if the current master fails

Where master-slave replication helps most:

- **Read-heavy datasets** — You can scale horizontally to handle more read traffic simply by adding more slave nodes and routing reads to them, while all writes still go to the single master.
- **Resilience for reads** — If the master fails, slaves can still serve read requests (though writes are blocked until a new master is appointed).

The key weakness — write bottleneck and inconsistency window:

Because there is only one master, it becomes a **bottleneck for writes** and a potential **single point of failure**. There is also an **inconsistency window**: because replication to slaves is not usually instantaneous, a slave might return stale data if it is read shortly after the master is updated but before the update has propagated.

Real-world intuition: A news website has far more readers than writers of content. Master-slave replication fits perfectly — editors write articles through the master, while millions of readers are served from many read-only slave replicas.

Exam Point: Master-slave replication is ideal for **read-scaling**, but all writes still funnel through a single master, which limits write throughput and creates a single point of failure for writes.

Peer-to-Peer Replication

Master-slave replication helps with read scalability, but it does not remove the write bottleneck or the single point of failure for writes. **Peer-to-peer replication** addresses this by treating **all nodes as equals** — any node can accept a write, and the nodes coordinate among themselves to keep their copies of the data synchronized.

Key characteristics:

Feature	Explanation
No single master	Every node can accept both reads and writes; there is no single point of failure for writes
Higher availability	Losing any one node does not stop the rest of the cluster from accepting writes
Risk of write-write conflicts	If two peers accept conflicting writes to the same data at the same time, the system must detect and resolve the conflict
Coordination overhead	Nodes must communicate to synchronize updates, which adds complexity compared to a single-master model

Real-world intuition: A globally distributed shopping cart service (such as Amazon's Dynamo-style systems) wants customers anywhere in the world to be able to add items to their cart without waiting on a single master server on another continent. Peer-to-peer replication allows writes to be accepted locally everywhere, at the cost of occasionally having to reconcile conflicting updates (for example, merging two carts if the same customer added items via two different replicas).

Exam Point: Peer-to-peer replication trades the simplicity of master-slave for greater **write availability** and no single point of failure — but it introduces the possibility of **write-write conflicts** that the application or database must resolve.

Master-Slave vs. Peer-to-Peer: Head-to-Head Comparison

Approach	How It Works	Trade-off
Master-Slave Replication	One master node handles all writes; slave nodes replicate from the master and can serve reads	Simple and avoids write-write conflicts, but the master is a write bottleneck and single point of failure
Peer-to-Peer Replication	All nodes are equal; any node accepts reads and writes; nodes synchronize with each other	Better write availability and no single point of failure, but must detect and resolve write-write conflicts

Combining Sharding and Replication

Sharding and replication are strategies that can be **combined** for both improved performance and improved resilience.

Master-slave replication + sharding:

If we combine master-slave replication with sharding, we end up with **multiple masters**, but each individual piece of data still has only **one** master responsible for it. Depending on configuration, a single node may act as master for some shards while acting as a slave for others, or nodes may be dedicated entirely to master or slave duties.

Peer-to-peer replication + sharding:

This combination is a common strategy for column-family databases such as Cassandra. In this setup, a cluster might have tens or even hundreds of nodes, with data sharded across them. A good starting point is a **replication factor of 3**, meaning each shard's data exists on three different nodes. If one node fails, the shards it held can still be rebuilt from the other two replicas.

Combination	Benefit
Sharding + Master-Slave Replication	Spreads both read/write load (via sharding) and provides read scalability plus fault tolerance for each shard (via replication)
Sharding + Peer-to-Peer Replication	Spreads load across many nodes while avoiding any single point of failure for writes on any shard — common in large column-family clusters

Real-world intuition: A large hotel-booking platform might shard bookings by region (Asia, Europe, Americas) for performance, and then replicate each regional shard across three data centers using peer-to-peer replication, so that a data-center outage in one region never causes total data loss for that region's bookings.

Exam Point: Sharding and replication solve **different** problems — sharding scales load, replication provides resilience and read scaling. Combining them is common in real-world, large-scale NoSQL deployments.

MODULE 2: CONSISTENCY

Why Consistency Becomes Complicated in NoSQL

Once you replicate or shard data across multiple nodes, you introduce the possibility that different clients see **different, conflicting versions** of the same data. NoSQL systems have to decide how strictly they will guarantee consistency, because stricter consistency generally comes at the cost of availability, latency, or throughput.

Update Consistency

Update consistency deals with what happens when **two clients try to update the same data at (or near) the same time** — a situation called a **write-write conflict**.

Write-write conflict: Client A and Client B both attempt to update the same resource with different values. Since the writes must be serialized (applied in some order), whichever write is applied second will overwrite the first — and the second write may have been based on data that was already out of date by the time it was applied.

Two broad strategies for handling write-write conflicts:

Approach	How It Works	Trade-off
Pessimistic	Prevents conflicts before they happen by taking a write lock on the resource; a	Sacrifices availability/responsiveness for

	second client must wait until the lock is released	safety — clients may be blocked waiting for a lock
Optimistic	Allows both clients to proceed, but detects conflicts before finally committing — often via a conditional update that checks whether the value has changed since it was read	Sacrifices some safety for availability/liveness — a client's update may fail and require a retry

A further complication in replicated systems: If two different nodes each receive one of the conflicting writes and process them in different orders, the nodes will end up disagreeing about the final value, and this disagreement must later be reconciled.

Real-world intuition: Two warehouse staff both try to update the stock count of the same product at the same time after separate deliveries arrive. If the system just lets whichever update reaches the database last "win," the earlier update's delivery quantity could be silently lost — a classic write-write conflict that update consistency mechanisms exist to prevent.

Exam Point: A write-write conflict happens when two updates to the same data collide. Pessimistic locking prevents the conflict outright (at the cost of availability); optimistic concurrency detects and resolves it after the fact (at the cost of possible retries).

Read Consistency

Even if writes never conflict with each other, a client can still see inconsistent data purely because of **how it reads** — this is called a **read-write conflict**. There are two distinct flavors.

Logical Consistency

Logical consistency is about a single client seeing a **coherent, related set of data** at once. Suppose Client A needs to update two related resources, X and Y. If Client A updates X but, before it can update Y, Client B reads both X and Y, then Client B sees an inconsistent in-between state — X reflects the new value, Y still reflects the old one.

In relational databases, this is normally solved with **transactions**, which apply both updates as a single atomic unit. In aggregate-oriented NoSQL databases, this guarantee is generally only available **within a single aggregate** — you can atomically update multiple fields of the same aggregate, but updating **multiple different aggregates** typically requires multiple separate operations, opening an **inconsistency window** during which another client might read a partially-updated state.

Replication Consistency

Replication consistency is about multiple clients seeing **different values for the same piece of data** because that data lives on multiple replicas that update at different times. If Client A updates resource X on one node, that update takes time to propagate to other replicas. A client reading from a replica that hasn't yet received the update will see a **stale** value — even though another client, reading the master or an already-updated replica at the very same moment, sees the new value.

A system exhibiting this behavior, where replicas will eventually converge to the same value, is described as **eventually consistent**.

Read-Your-Writes / Session Consistency

Even in an eventually consistent system, it is often important that a client can immediately see the effect of **its own** writes, even if other clients temporarily see stale data. This weaker but very useful guarantee is called **read-your-writes consistency** (or session consistency). A common technique for achieving it is **sticky sessions** (also called session affinity) — routing all of a particular user's requests to the same node for the duration of their session, so they always see their own latest updates.

Update Consistency vs. Read Consistency: Comparison

Aspect	Update Consistency	Read Consistency
Conflict type	Write-write conflict — two writers collide	Read-write conflict — a reader sees an inconsistent or stale state
Main causes	Concurrent updates to the same resource	Reading across related resources mid-update (logical), or reading a stale replica (replication)
Typical fixes	Pessimistic locking or optimistic conditional updates	Transactions/aggregate atomicity (logical); eventual consistency and sticky sessions (replication)

Real-world intuition: After adding an item to your online shopping cart, you expect to immediately see it in your cart (read-your-writes) — even if it takes a few seconds for that update to reach every server in the company's data centers, so that a customer-service rep looking at a different replica might briefly see your cart as it was before.

Exam Point: Read-your-writes consistency is one of the most commonly demanded consistency guarantees in real applications — users are far more tolerant of other people's data being briefly stale than they are of their own recent actions appearing to have been lost.

Relaxing Consistency

Strong consistency — where all nodes always agree and every read returns the latest write — is appealing, but it is expensive to achieve in a distributed system: it typically requires contacting multiple (or all) nodes before responding, which increases latency and reduces availability during network problems. NoSQL systems often deliberately **relax** consistency to gain availability, performance, or both.

Certain inconsistencies can be tolerated by system design. For example, allowing a flight to be briefly overbooked by a passenger or two, or allowing the same item to be added to a shopping cart from two devices, may be perfectly manageable if the application has a sensible way to reconcile the conflict later (bump a passenger with compensation, merge the cart contents at checkout).

The CAP Theorem

The CAP theorem, formulated by Eric Brewer, relates three properties of a distributed system:

Property	Meaning
Consistency (C)	Every read receives the most recent write (or an error) — all nodes see the same data at the same time
Availability (A)	Every request received by a non-failing node must result in a (non-error) response
Partition Tolerance (P)	The system continues to operate even if network communication is broken (partitioned) between sets of nodes

The theorem, in its simplified form: Given these three properties, a distributed system can only guarantee **two of the three** at the same time. Since network partitions are always a possibility in any real distributed system, partition tolerance is effectively mandatory — so, in practice, the real choice a system architect faces during a partition is between **consistency** and **availability**.

Working through the trade-off: Imagine two nodes, A and B. If a write arrives at node A and the system insists on consistency, A must first confirm with B that the write is safe before accepting it. If the link between A and B breaks, A cannot safely accept the write — this sacrifices **availability**. Alternatively, if A accepts the write without waiting for B, the system remains **available**, but if B also accepts a conflicting write during the same partition, the two nodes become **inconsistent** until the partition heals and the conflict is reconciled.

CP vs. AP systems:

System Type	Behavior During a Partition	Example Tendency
CP (Consistency + Partition Tolerance)	Refuses requests it cannot guarantee are consistent, sacrificing availability	HBase, MongoDB (in many configurations)
AP (Availability + Partition Tolerance)	Keeps accepting reads and writes on both sides of the partition, sacrificing strict consistency (becomes eventually consistent)	Cassandra, DynamoDB, CouchDB (in many configurations)

Real-world intuition: During a network split between two data centers, a CP-style banking ledger might refuse withdrawal requests it cannot verify are safe (protecting correctness), while an AP-style social-media "like" counter would rather keep accepting likes on both sides and reconcile the count later (protecting availability), since being briefly wrong about a like count matters far less than an account balance being wrong.

Exam Point: The CAP theorem only becomes relevant when there IS a network partition. It does not say you must always sacrifice consistency — only that **when** a partition occurs, you must choose between availability and consistency for the affected requests.

Relaxing Durability

Durability means that once a write is confirmed, it will survive subsequent failures (crashes, power loss, etc.). Like consistency, durability can be relaxed to improve performance — usually by trading it off against **latency**.

How durability is commonly relaxed:

- In-memory writes** — Some databases (e.g., Redis in certain configurations) can be configured to acknowledge a write as soon as it is in memory, before it is flushed to disk, in exchange for much faster write response times. If the server crashes before the flush, that write is lost.
- Replication as a durability substitute** — Rather than waiting for a slow disk write, a system may consider a write "durable

enough" once it has been replicated to memory on a couple of other nodes — the chance of all of them failing at once is low, even though technically none of them has persisted to disk yet.

Real-world intuition: A high-frequency analytics/logging pipeline that records millions of low-value events per second might deliberately relax durability — losing a handful of log entries during a rare crash is an acceptable trade for dramatically higher write throughput; a banking transaction system would not make the same trade.

Exam Point: Durability can be traded off against latency, especially when using replication: as long as a write reaches a couple of other nodes, the risk of losing it is often considered acceptably low, even before it's written to disk.

Quorums

When data is replicated, we need a way to decide how many nodes must participate in a read or write before we can trust the result. This is governed by **quorums**, defined using three key numbers:

Term	Meaning
N	The replication factor — the total number of nodes/replicas that hold a copy of a given piece of data
W	The write quorum — the number of replicas that must acknowledge a write before it is considered successful
R	The read quorum — the number of replicas that must be contacted (and compared) for a read before the result is trusted

The key formulas:

- To avoid write-write conflicts and ensure strong **write** consistency: $W > N / 2$ (a majority of nodes must agree on a write).
- To avoid read-write conflicts and ensure strong **read** consistency: $R + W > N$.

The intuition behind $R + W > N$ is simple: if the set of nodes you write to and the set of nodes you read from are guaranteed to **overlap** by at least one node, then any read is guaranteed to include at least one copy of the most recent write.

Worked example with N = 3 (a very common configuration):

W	R	R + W vs. N	Outcome
2	2	$4 > 3$	Balanced read/write speed; strong consistency guaranteed
3	1	$4 > 3$	Very fast reads, slower writes (all replicas must confirm); no write can succeed if even one node is down
1	3	$4 > 3$	Very fast writes, slower reads (all replicas must be checked); strong consistency, but writes are fragile if used carelessly
2	1	$3 = 3$ (not $>$)	Fast reads and writes, but stale reads are possible — the read node might not be one of the two written nodes

Real-world intuition — hotel room booking: A hotel-chain booking system with $N = 3$ replicas per room record might use $W = 2$, $R = 2$ to guarantee that once a room is marked "booked," any subsequent read (from any front desk or app) is guaranteed to see that booking and avoid a double-booking — at the modest cost of needing two nodes to respond for every read and every write, rather than just one.

Exam Point: Memorize the two quorum formulas: $W > N/2$ prevents write-write conflicts; $R + W > N$ guarantees a read will always see the latest write. These formulas are the mathematical foundation of "tunable consistency" in many NoSQL databases (e.g., Cassandra, Riak, DynamoDB-style systems).

MODULE 3: VERSION STAMPS & TRANSACTIONS

Why Version Stamps Are Needed

A common criticism of NoSQL databases is their limited support for multi-record **transactions** compared to relational databases. Transactions are a useful tool for maintaining consistency, since they let a group of changes succeed or fail as a single unit. However, aggregate-oriented NoSQL databases do support **atomic updates within a single aggregate**, since an aggregate is deliberately designed to be the natural unit of update — reducing (though not eliminating) the need for cross-record transactions in many designs.

Even in fully transactional relational systems, transactions have limits — some real-world processes require human decision-making steps that simply cannot be wrapped inside a fast, short-lived database transaction. This is where the distinction between **business transactions** and **system transactions** becomes important, and where **version stamps** come in as a lightweight tool to detect conflicts without full locking.

Business and System Transactions

Not all "transactions" that matter to an organization map neatly onto a single database transaction. It helps to distinguish between two levels:

Type	Definition	Lock Duration
System Transaction	A technical transaction handled by the database engine itself — the application only begins a true database (system) transaction at the point it is ready to commit its interaction with the user	Short — locks are held only briefly, for the duration of the actual database operation
Business Transaction	A real-world, multi-step interaction that may span a long period of human decision-making — for example, a customer filling out a multi-page order form over several minutes or even browsing across multiple sessions	Long — if implemented naively with database locks held for the whole interaction, this would tie up resources for far too long

The core problem this creates: Because locks are held for a long time in a naive "business transaction as a single database transaction" design, but only for a short time in an actual system transaction, **the data can change out from under the user** between when they started their business transaction and when they finally submit it. Calculations and decisions the user made may have been based on data that is now stale.

A concrete real-world example: A customer builds an order over several minutes. While they are deciding, the price list changes (a product's price is updated), or the customer's saved shipping address is updated by another process, altering the shipping charges. By the time the customer submits the order, the totals they saw on screen may no longer match the current data.

Why this matters: Since we generally cannot (and should not) hold real database locks for the entire duration of a long, human-paced business transaction, we need some lightweight way to detect whether the underlying data changed while the user was deciding — this is exactly the problem **version stamps** solve.

Exam Point: A business transaction can span minutes or hours of human interaction, while a system (database) transaction is short and technical. Holding real database locks for an entire business transaction is impractical — version stamps offer a way to detect conflicts without doing so.

Version Stamps

A **version stamp** is a field attached to a record that **changes every time the underlying data changes**. It can be implemented in different ways: a counter that increments on each update, a timestamp, a content hash, or a globally unique random value generated on each change.

How version stamps are used to detect conflicts (conditional/optimistic update):

1. When a client reads a record, it also reads and remembers the record's current version stamp.
2. When the client later wants to write an update, it sends the update along with the version stamp it originally read.
3. The database compares the submitted version stamp against the record's **current** version stamp.
4. If they match, no one else has changed the record in the meantime — the write proceeds, and a new version stamp is generated.
5. If they don't match, someone else has updated the record since it was read — the write is rejected, and the client must re-read the data and decide how to proceed (retry, merge, or alert the user).

Real-world intuition: This is exactly how many web applications prevent "lost updates" on editable forms — a hidden field stores the version of the record when the edit page was loaded; if someone else has saved changes in the meantime, the version numbers won't match and the second save is rejected with a "this record has changed, please refresh" message, instead of silently overwriting the first person's edit.

Exam Point: A version stamp turns a write-write conflict detection problem into a simple equality check — it is a lightweight, optimistic alternative to holding a lock for the entire duration of a business transaction.

Version Stamps on Multiple Nodes

A single incrementing counter works well on a single server, but it breaks down once data is **replicated across multiple nodes** — especially in a peer-to-peer replication setup where more than one node can accept writes independently. If two different nodes each generate their own "next" version number for concurrent updates, a simple counter cannot reliably tell which version is actually newer, or whether the two updates conflict.

Approaches to version stamps in a distributed setting:

Approach	How It Works	Limitation
----------	--------------	------------

Simple counter per node	Each node maintains its own counter	Cannot tell whether two versions from different nodes are the same, one is newer, or they conflict
Timestamps	Each update is stamped with the time it occurred	Requires closely synchronized clocks across nodes, which is difficult to guarantee in distributed systems; clock drift can cause incorrect ordering
Vector Stamps (Vector Clocks)	Each node keeps a small counter for itself within the version stamp; the stamp becomes a set of (node, counter) pairs, e.g. [(NodeA, 2), (NodeB, 1)]	More complex to implement and reason about, but correctly detects both "one version is a direct descendant of another" and "these two versions were made independently and now conflict"

How vector stamps detect conflicts:

- Each node increments only **its own** counter within the vector stamp whenever it processes an update.
- One version is considered an **ancestor** of another if, for every node's counter in the first version, the second version's counter for that same node is equal or higher (i.e., the second version has "seen" everything the first version had).
- If neither version is an ancestor of the other — meaning each has counters the other doesn't have or hasn't caught up to — the two versions are on **parallel, conflicting branches**, and the conflict must be resolved (often by the application, using domain-specific logic).

Real-world intuition: This is essentially how distributed shopping-cart or file-sync systems (such as Amazon's Dynamo or Git itself, conceptually) determine whether two versions of the same item are safely one-after-another, or whether they diverged and need to be merged — for example, merging two versions of a shopping cart that were each updated on a different, temporarily disconnected replica.

Exam Point: A simple counter or raw timestamp is not reliable for detecting conflicts across multiple writable nodes. **Vector stamps** solve this by tracking a separate counter per node, allowing the system to distinguish a clean sequential update from a true concurrent conflict that needs reconciliation.

MODULE 4: UNIT 2 SUMMARY & EXAM PREPARATION

4.1 Key Definitions to Memorize

Term	Definition
Sharding	Distributing different data across multiple servers so each server holds a distinct subset of the data
Replication	Copying the same data across multiple servers so it exists in more than one place
Master-Slave Replication	One authoritative master node handles all writes; slave nodes replicate from it and can serve reads
Peer-to-Peer Replication	All nodes are equal and can accept both reads and writes; nodes synchronize with each other
Write-Write Conflict	Two clients attempt to update the same resource with different values at the same time
Read-Write Conflict	A client reads inconsistent or stale data because of a concurrent update elsewhere in the system
Eventual Consistency	A guarantee that, given enough time with no new updates, all replicas will converge to the same value
Read-Your-Writes Consistency	A guarantee that a client can always see the effects of its own most recent writes
CAP Theorem	A distributed system can guarantee at most two of Consistency, Availability, and Partition Tolerance at once
Quorum	The minimum number of replicas that must participate in a read or write for the result to be trusted
Business Transaction	A long-running, human-paced, multi-step real-world interaction that does not map directly to a single database transaction
System Transaction	A short, technical database transaction handled directly by the database engine
Version Stamp	A field that changes every time a record's underlying data changes, used to detect conflicting updates
Vector Stamp (Vector Clock)	A version stamp made of per-node counters, used to detect conflicts across multiple writable replicas

4.2 Comparison Tables to Memorize

Distribution Models at a Glance:

Model	Solves	Weakness
-------	--------	----------

Single Server	Simplicity — no distribution complexity at all	No horizontal scaling, no built-in redundancy
Sharding	Read/write load scaling by splitting data across nodes	No improvement in resilience by itself
Master-Slave Replication	Read scaling and read resilience	Write bottleneck and single point of failure for writes
Peer-to-Peer Replication	Write availability — no single point of failure	Possible write-write conflicts requiring reconciliation
Sharding + Replication	Scales load AND provides resilience together	Highest operational complexity

CAP Theorem Trade-off Table:

Choice During a Partition	What You Keep	What You Sacrifice	Example Systems
CP (Consistency + Partition Tolerance)	All nodes agree on data; no stale reads	Availability — some requests may be refused	HBase, MongoDB (typical config)
AP (Availability + Partition Tolerance)	System keeps responding to all requests	Strong consistency — reads may be stale until convergence	Cassandra, DynamoDB, CouchDB (typical config)

Quorum Formulas to Memorize:

Goal	Formula	Meaning
Prevent write-write conflicts	W > N / 2	A write must be acknowledged by a majority of replicas
Guarantee strong read consistency	R + W > N	The read set and write set are guaranteed to overlap by at least one node

Update Consistency vs. Read Consistency (recap):

Aspect	Update Consistency	Read Consistency
Problem	Write-write conflict	Read-write conflict (logical or replication-based)
Common fix	Pessimistic locks or optimistic conditional updates (version stamps)	Transactions/aggregate atomicity; sticky sessions for read-your-writes